



UNIVERSITY OF MAURITIUS

Faculty of Information, Communication
and Digital Technologies

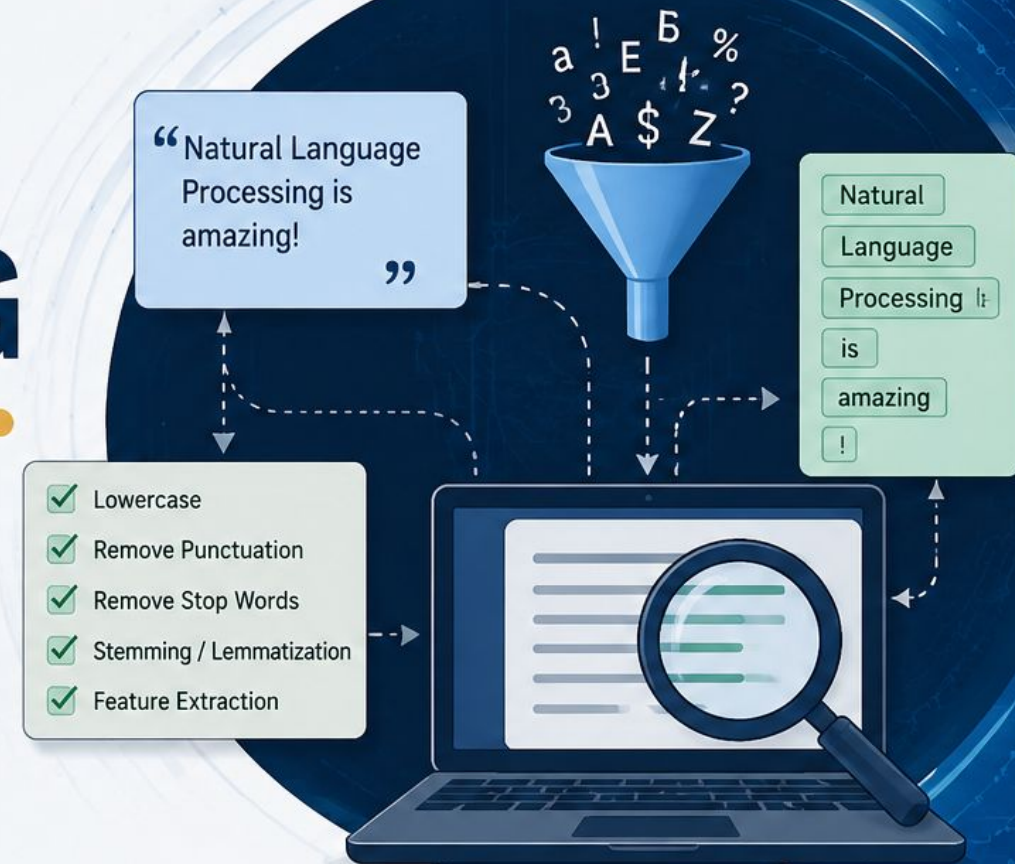
WEEK 2

TEXT PROCESSING

Cleaning, Tokenisation, Stop Words,
Stemming, Lemmatisation and
Feature Extraction

SIS3204Y(5)

Natural Language Processing



Understand
text preprocessing
techniques



Transform
raw text into
meaningful data



Build
the foundation for
effective NLP models

*"Well-processed text is the
foundation of intelligent language
understanding."*



LEARNING OUTCOMES

By the end of this lecture, you should be able to:

- ✓ Explain the **purpose and importance** of text processing in Natural Language Processing.
- ✓ Apply **text cleaning** techniques to prepare raw text for analysis.
- ✓ Perform **tokenisation** using different tokenisation methods.
- ✓ Identify and remove **stop words** and discuss their impact on text analysis.
- ✓ Distinguish between **stemming** and **lemmatisation** and apply them to words.
- ✓ Convert text into numerical features using basic **feature extraction** techniques.
- ✓ Demonstrate an **end-to-end text processing workflow** using Python.



TEXT PROCESSING

The foundation for all NLP tasks!



Good preprocessing leads to
better models and **better results**!

FROM HUMAN TEXT TO MACHINE-PROCESSABLE DATA

Computers cannot directly understand the rich, messy and unstructured text produced by humans.

Text processing is the bridge that converts human language into a form that machines can analyse and learn from.



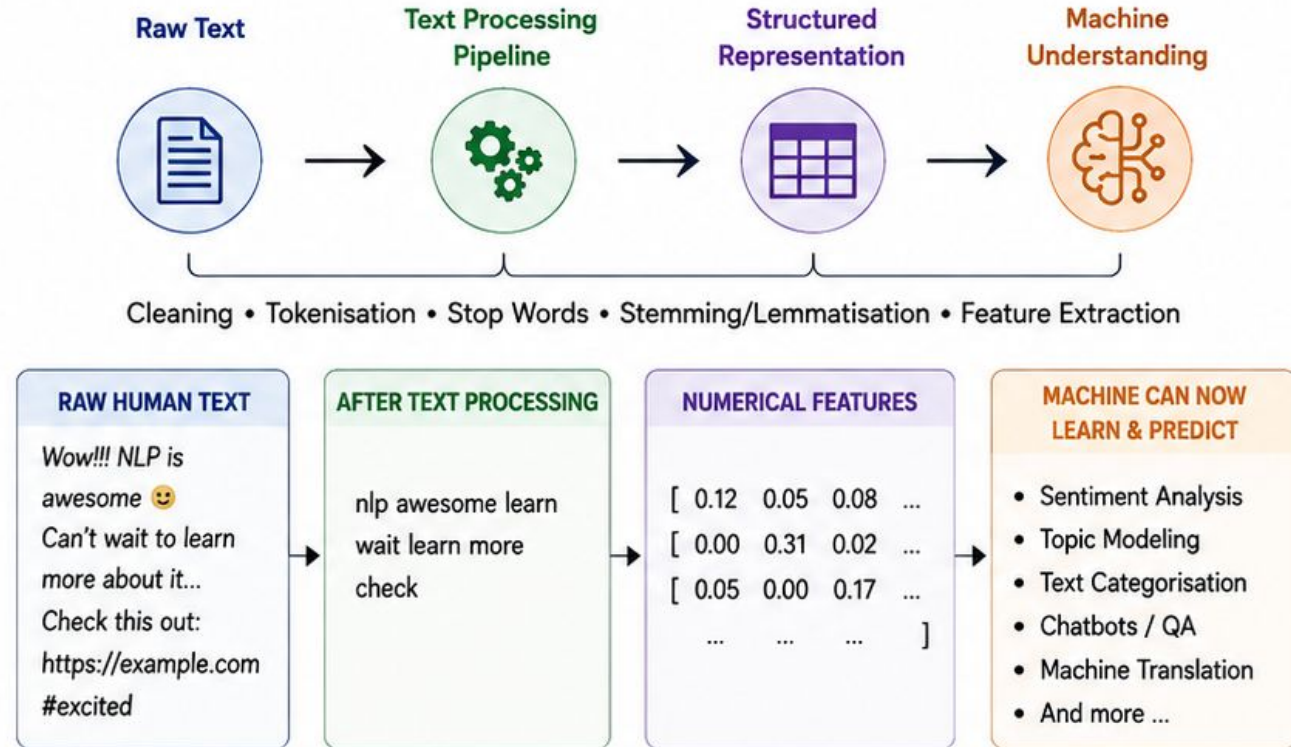
1. Humans communicate in natural language
It is flexible, context-dependent and often contains noise, inconsistencies, and ambiguity.



2. Computers need structured, numerical representations
Machines work with numbers and patterns, not raw text.



3. Text processing is the bridge
It cleans, transforms and represents text in a structured way that machines can process and learn from.



KEY TAKEAWAY

Before a computer can analyse or learn from text, the text must be cleaned, transformed and represented in a structured way.

WHY RAW TEXT IS MESSY

Text that humans write or speak is full of variations and irregularities. These make it **difficult for computers to understand and analyse directly**.



KEY IDEA

Raw text is rich in meaning for humans, but it is noisy and inconsistent for machines.

1. Inconsistent Capitalisation



Same words written in different cases.

Example

NLP is fun.
nlp is fun.
Nlp Is Fun.
NLp IS FUN.

2. Punctuation & Symbols



Extra symbols may not carry meaning for computers.

Example

Wow!!! NLP is great!!!
Is NLP great?
NLP... is great 😊
(NLP) is great.

3. Numbers



Numbers can appear in many formats.

Example

We scored 10/10.
The score is ten.
10 out of 10.
Score: 10.0

4. URLs, Emails & Mentions



Links and handles are often not useful for analysis.

Example

Visit <https://uom.ac.mu>
Email me at info@uom.ac.mu
Thanks @john_doe!

5. Spelling Errors & Slang



Typos, slang and informal words add noise.

Example

I luv NLP so much!
It's awsum & gr8.
Gon'na learn more.
Thnx a lot!

6. Extra Spaces & Line Breaks



Irregular spacing and line breaks create inconsistency.

Example

NLP is powerful.
NLP is
powerful.
NLP is powerful.



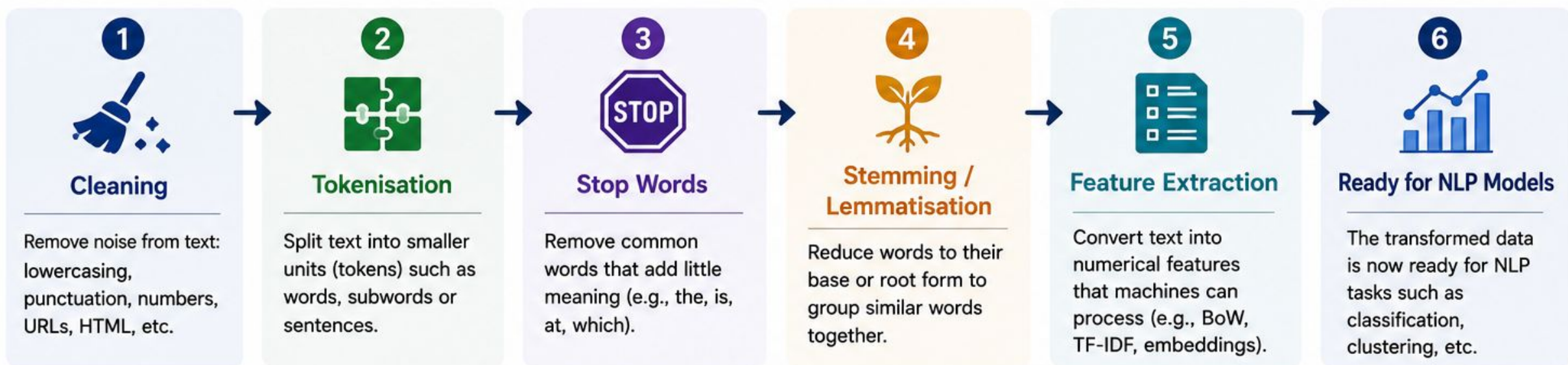
Takeaway

Before we can analyse text, we must **clean** and **standardise** it so that machines can focus on what really matters: the **meaning**.



THE TEXT PREPROCESSING PIPELINE

Text preprocessing converts raw, messy text into clean, structured and meaningful representations. It is a **sequence of steps** where the output of one step becomes the input of the next.



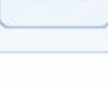
KEY TAKEAWAY

The better the preprocessing, the better the quality of data we give to our models, and the better the performance of our NLP systems.

WHAT IS TEXT CLEANING?

Text cleaning (also called **text normalization**) is the first step in preprocessing. It removes or standardises unwanted, noisy or inconsistent elements from raw text.

Goal: convert messy text into a clean and consistent form.

Raw / Messy Text	After Cleaning	What Was Cleaned?
 This IS nLp!!!	this is nlp	Lowercasing, removed punctuation
 Visit https://uom.ac.mu for more info.	visit for more info	Removed URL
 Email me at: john.doe@uom.ac.mu	email me at	Removed email address
 I scored 10/10 in 2024!	i scored in	Removed numbers
 #excited @AI_NLP 😊	excited ai nlp	Removed hashtags, mentions, emojis
 Great product , really loved it !!!	great product really loved it	Extra spaces, punctuation normalised



WHY CLEAN TEXT?

- ✓ Reduces noise and inconsistencies
- ✓ Improves model accuracy
- ✓ Helps in better pattern extraction
- ✓ Makes text data consistent and reliable

COMMON CLEANING TASKS

-  Convert to lowercase
-  Remove punctuation and special characters
-  Remove or standardise numbers
-  Remove URLs, emails, HTML tags
-  Remove hashtags, mentions, emojis
-  Remove extra spaces and line breaks



REMEMBER

Do not remove information that could be important for your task (e.g., numbers in product reviews, names in NER).



KEY TAKEAWAY

Text cleaning prepares the data by removing noise and standardising text, so that the next steps in the pipeline can work more effectively.

CONVERTING TEXT TO LOWERCASE

Lowercasing converts all uppercase letters to lowercase.

This ensures consistency so that different cases of the same word are treated as the same token.

BEFORE (Raw Text)		AFTER (Lowercased Text)
 This Is NLP!	→	this is nlp!
 I Love Natural Language Processing.	→	i love natural language processing.
 Send Email to Info@UoM.ac.mu	→	send email to info@uom.ac.mu
 #AI_NLP is Awesome	→	#ai_nlp is awesome
 NLP Models Are Getting Better Every Day.	→	nlp models are getting better every day.



EXAMPLE IN CONTEXT

Without lowercasing, the following would be treated as different words:

NLP Nlp nlp NLP

With lowercasing, all map to: **nlp**



WHY IT MATTERS

- ✓ Reduces vocabulary size
- ✓ Improves matching and retrieval
- ✓ Helps models generalise better



MINI DEMO (Python)

```
text = "This Is NLP!\nSend Email to  
Info@UoM.ac.mu"  
text_lower = text.lower()  
print(text_lower)
```

Output:

```
this is nlp!  
send email to info@uom.ac.mu
```



NOTE

Lowercasing may not be suitable for all tasks (e.g., Named Entity Recognition where case carries meaning).



KEY TAKEAWAY

Lowercasing is a simple but powerful step that reduces variations and makes text data **consistent** for further processing.

HANDLING SPELLING, CONTRACTIONS AND INFORMAL LANGUAGE

Real-world text is often noisy. People make spelling mistakes, use contractions and write in informal ways.

NLP systems understand text better when we **standardise** and **normalise** it.



1. Spelling Normalisation

Fix common spelling mistakes to the correct form.

EXAMPLE

Input: recievd the application.

Output: received the application.



Tools & Tips

- Spell checker (e.g., PySpellChecker)
- Domain dictionary
- Edit distance / context-aware correction (e.g., SymSpell)



2. Expanding Contractions

Expand contracted forms to their full versions.

EXAMPLE

Input: I'm happy, she's not here.

Output: I am happy, she is not here.



Common Contractions

- | | |
|------------------|--------------------|
| • I'm → I am | • won't → will not |
| • it's → it is | • didn't → did not |
| • don't → do not | • she's → she is |
| • can't → cannot | • we're → we are |



Expand contractions before tokenisation to avoid splitting (e.g., "don't" → "don", "t").



3. Handling Informal Language

Convert informal language, slang and abbreviations to standard form.

EXAMPLE

Input: u r goin 2 the mtg tmr?

Output: you are going to the meeting tomorrow?



Common Informal Forms

- | | |
|----------------|-------------------------|
| • u → you | • mtg → meeting |
| • r → are | • tmr / tomo → tomorrow |
| • goin → going | • lol → laugh out loud |
| • 2 → to | • btw → by the way |



Use a mapping list or glossary. Consider context before normalising.



BEST PRACTICES

- ✓ Use a domain-specific dictionary when possible.
- ✓ Be careful not to over-correct proper nouns.
- ✓ Expand contractions before tokenisation.
- ✓ Keep a mapping list for informal terms.
- ✓ Consider context before correcting spelling.
- ✓ Log changes for transparency and reproducibility.



REMEMBER

Standardising noisy text reduces noise and increases the quality of the data your NLP model learns from.



GOAL: Make text cleaner and easier for machines to understand by fixing spelling, expanding contractions and normalising informal language.

Raw Text
(Noisy)



Clean Text
(Standardised)

REMOVING PUNCTUATION & SPECIAL CHARACTERS

Punctuation and special characters add **noise** and do not usually contribute to the meaning of text in most NLP tasks.

Removing them helps models focus on the **important words**.

Goal: to clean text by removing or replacing punctuation and special characters.



WHY IT MATTERS

- ✓ Reduces noise and inconsistency
- ✓ Improves tokenisation results
- ✓ Prevents false distinctions
- ✓ Enhances model performance

EXAMPLES: BEFORE AND AFTER REMOVING PUNCTUATION			
	Raw Text (Before)	After Removing Punctuation	What Was Removed?
	Wow!!! NLP is awesome!!!	Wow NLP is awesome	Exclamation marks (!!!)
	Email me at: info@uom.ac.mu.	Email me at info@uom ac mu	Colon (:) and period (.)
	Check out #NLP, it's amazing!	Check out NLP its amazing	Hashtag (#), comma (,), apostrophe (')
	Visit (https://uom.ac.mu) today.	Visit https uom ac mu today	Parentheses (()), colon in URL, dots (.)
	C++ is powerful; isn't it?	C is powerful isnt it	Plus signs (++), semicolon (;), apostrophe ('), question mark (?)
	Great product :) really loved it!	Great product really loved it	Emoticon (:) and exclamation mark (!)

COMMON PUNCTUATION & SPECIAL CHARACTERS

-  **Punctuation Marks**
., ; : ! ? ' " () [] { } ... - _
-  **Special Symbols**
@ # \$ % ^ & * + = < > / \ | ~ `
-  **Emojis & Emoticons**
😊 😄 🚀 👍 😊
-  **Other**
New lines (\n), Tabs (\t), Multiple spaces



TIP

For specific tasks like sentiment analysis, exclamation marks (e.g., "!!") can carry emotion, so decide based on the use case.



MINI DEMO (Python)

```
import string
text = "Wow!!! NLP is awesome :)"
text_clean = text.translate(str.maketrans('', '', string.punctuation))
print(text_clean)           # Output: Wow NLP is awesome
```



KEY TAKEAWAY

Removing punctuation and special characters cleans the text, reduces noise, and helps models learn **better patterns** from the important content.

BEFORE VS AFTER TEXT CLEANING

Text cleaning transforms noisy, unstructured text into **clean, consistent** text that is easier for NLP models to process and understand.

STEP	BEFORE CLEANING (RAW TEXT)	AFTER CLEANING (CLEAN TEXT)
 Example Text	I'm sooo happy!! 😊 U guys r awesome... thx a lot!!! Can't wait 2 c u all @ the meetup tmrw ;)	i am so happy you guys are awesome thank you a lot cannot wait to see you all at the meetup tomorrow
 Lowercasing	I'm SOOO Happy!! 😊 U Guys R AWESOME... Thx A Lot!!! Can't Wait 2 C U All @ The Meetup Tmrw ;)	i am sooo happy!! 😊 u guys r awesome... thx a lot!!! can't wait 2 c u all @ the meetup tmrw ;)
 Contraction Expansion	i'm sooo happy!! 😊 u guys r awesome... thx a lot!!! can't wait 2 c u all @ the meetup tmrw ;)	i am sooo happy!! 😊 you guys are awesome... thank you a lot!!! cannot wait to see you all @ the meetup tomorrow ;)
 Tokenisation	i am sooo happy!! 😊 you guys are awesome... thank you a lot!!! cannot wait to see you all @ the meetup tomorrow ;)	[i, am, sooo, happy, !!, 😊, you, guys, are, awesome, thank, you, a, lot, !!!, cannot, wait, to, see, you, all, @, the, meetup, tomorrow, ;)]
 Stop Word Removal	[i, am, sooo, happy, !!, 😊, you, guys, are, awesome, thank, you, a, lot, !!!, cannot, wait, to, see, you, all, @, the, meetup, tomorrow, ;)]	[sooo, happy, !!, 😊, guys, awesome, thank, lot, !!!, cannot, wait, see,, meetup, tomorrow, ;)]
 Final Cleaned Text	(Raw text is noisy and inconsistent)	clean, normalised and consistent text



BENEFITS OF CLEANED TEXT

- ✓ Reduces noise and ambiguity
- ✓ Improves model accuracy
- ✓ Speeds up training and inference
- ✓ Enhances feature extraction
- ✓ Supports better decision making



KEY TAKEAWAY

Cleaning text is not about changing the meaning, it's about removing the noise so that the meaning can be learned better by models.



NOTE

Good cleaning today =
better insights tomorrow!



KEY TAKEAWAY

Clean data leads to **clear** understanding. **Clean** text leads to **smart** models.

TOKENISATION: SPLITTING TEXT INTO UNITS

Tokenisation is the process of breaking text into **smaller units** called **tokens**.

Tokens can be words, subwords or characters depending on the method used.

Goal: to create meaningful, consistent units that can be processed by machines.

EXAMPLE: TOKENISING A SENTENCE



NOTE: Different tokenisers may keep punctuation or use subword splitting (e.g., “tokenisation” → “token”, “##isation”). The choice of tokeniser can affect vocabulary size and model performance.



KEY TAKEAWAY

Tokenisation breaks text into smaller, meaningful units (**tokens**) so that computers can understand and process it effectively.



WHY IT MATTERS

- ✓ Prepares text for analysis
- ✓ Enables counting and statistics
- ✓ Essential for all NLP models
- ✓ Improves consistency and accuracy



COMMON TOKENISATION METHODS

- **Whitespace Tokenisation**
Split by spaces.
- **Word Tokenisation**
Split into words and remove punctuation.
- **Subword Tokenisation (e.g., BPE, WordPiece)**
Split words into subwords for better handling of rare or unknown words.
- **Character Tokenisation**
Split into individual characters.



KEY MEMORY

Tokenisation turns raw text into a sequence of tokens — the building blocks for all NLP tasks.

WORD TOKENISATION

Tokenisation is the process of breaking text into smaller units called **tokens** (usually words). It is the first step in most NLP pipelines.



Raw Text (Input):

"I'm soo happy!! 😊 You guys are AWESOME... Can't wait 2 c u all @ the meetup tmrw ;)"

TOKENISATION STEPS

STEP	DESCRIPTION	OUTPUT (TOKENS)
1 Raw Text	Original text as it is.	"I'm soo happy!! 😊 You guys are AWESOME... Can't wait 2 c u all @ the meetup tmrw ;)"
2 Basic Tokenisation (Split by spaces)	Split text into tokens using spaces as the separator.	["I'm", "soo", "happy!!", "😊", "You", "guys", "are", "AWESOME...", "Can't", "wait", "2", "c", "u", "all", "@", "the", "meetup", "tmrw", ";)"]
3 Punctuation Separation	Separate punctuation from words.	["I'm", "soo", "happy", "!", "!", "😊", "You", "guys", "are", "AWESOME", "...", "Can't", "wait", "2", "c", "u", "all", "@", "the", "meetup", "tmrw", ";", ")"]
4 Normalisation (Case, Lowercasing)	Convert tokens to a common case (e.g., lowercase).	["i'm", "soo", "happy", "!", "!", "😊", "you", "guys", "are", "awesome", "...", "can't", "wait", "2", "c", "u", "all", "@", "the", "meetup", "tmrw", ";", ")"]
5 Final Tokens	Cleaned list of tokens ready for further processing.	["i'm", "soo", "happy", "!", "!", "😊", "you", "guys", "are", "awesome", "...", "can't", "wait", "2", "c", "u", "all", "@", "the", "meetup", "tmrw", ";", ")"]



KEY TAKEAWAY

Tokenisation converts raw text into **manageable pieces** that machines can understand and process.



BEST PRACTICES

- ✓ Handle punctuation carefully.
- ✓ Decide how to treat emojis and emoticons.
- ✓ Keep contractions (e.g., can't) or expand them based on your task.
- ✓ Use consistent rules for your entire dataset.
- ✓ Tokenisation affects downstream tasks – choose the right strategy for your use case.



COMMON CHOICES

- Whitespace Tokeniser (basic)
- Regex-based Tokeniser
- Library Tokenisers:
 - NLTK (word_tokenize)
 - spaCy
 - Hugging Face (AutoTokenizer)



GOAL

Break text into meaningful tokens so that the model can learn from the structure and patterns in the data.

SENTENCE TOKENISATION

Sentence tokenisation is the process of splitting a paragraph or document into individual sentences. It helps NLP systems understand the **structure** and **boundaries** of text.



Example Raw Text (Input):

"I'm soo happy!! 😊 You guys are AWESOME. Can't wait 2 c u all @ the meetup tmrw ;)"
Let's finalize the agenda. Please confirm by Friday."
Thanks & see you there!"

SENTENCE TOKENISATION STEPS

STEP	DESCRIPTION	OUTPUT (SENTENCES)
1 Raw Text	Original text as it is.	"I'm soo happy!! 😊 You guys are AWESOME. Can't wait 2 c u all @ the meetup tmrw ;)" Let's finalize the agenda. Please confirm by Friday. Thanks & see you there!"
2 Detect Sentence Boundaries	Identify the end of sentences using punctuation (. ! ?) and context rules.	Boundaries found after: "AWESOME." ";" "agenda." "Friday." "there!"
3 Split into Sentences	Split the text at the detected boundaries.	1. I'm soo happy!! 😊 You guys are AWESOME. 2. Can't wait 2 c u all @ the meetup tmrw ;) 3. Let's finalize the agenda. 4. Please confirm by Friday. 5. Thanks & see you there!"
4 Clean Sentences (Optional)	Trim whitespace and normalize spacing.	1. I'm soo happy!! 😊 You guys are AWESOME. 2. Can't wait 2 c u all @ the meetup tmrw ;) 3. Let's finalize the agenda. 4. Please confirm by Friday. 5. Thanks & see you there!"
5 Final Sentences	List of clean sentences ready for further processing.	List of 5 clean sentences (as above).



KEY TAKEAWAY

Sentence tokenisation gives **structure** to text by identifying where **one idea ends** and **another begins**.



BEST PRACTICES

- ✓ Use punctuation (. ! ?) as primary cues, but consider context.
- ✓ Handle abbreviations (e.g., Dr., Mr., U.S.A.) carefully.
- ✓ Handle quotes, brackets and ellipses (...), URLs, and emails.
- ✓ Be consistent across your dataset.
- ✓ Review edge cases for your domain (e.g., questions, lists).



COMMON CHALLENGES

- Abbreviations causing wrong splits.
- Exclamations and questions.
- Ellipses (...) and multiple punctuation.
- Newlines vs. real sentence breaks.
- Domain-specific writing styles.



GOAL

Accurately split text into sentences so models can understand context, structure and meaning more effectively.

SUBWORD TOKENISATION

Subword tokenisation breaks words into smaller meaningful units (subwords or pieces) instead of whole words. This helps NLP models handle **rare**, **unknown** or **complex words** more effectively.



Example Raw Text (Input):
“The internationalization of user-friendly interfaces helps organizations communicate better.”

SUBWORD TOKENISATION STEPS (using WordPiece / BPE as examples)			
STEP	DESCRIPTION	EXAMPLE	OUTPUT (SUBWORDS)
1 Raw Text	Original text as input.	The internationalization of user-friendly interfaces ...	[the, internationalization, of, user-friendly, interfaces, helps, ...]
2 Apply Subword Algorithm	Apply a subword model (e.g., WordPiece, BPE) to split words into subword units.	The internationalization of user-friendly interfaces ...	[the, international, ##ization, of, user, -, friendly, interfaces, helps, ...]
3 Handle Unknown Words	Break rare/unknown words into known subword pieces using the algorithm's rules.	The internationalization of user-friendly interfaces ...	[the, inter, ##national, ##ization, of, user, -, friendly, inter, ##faces, helps, ...]
4 Add Special Tokens (Optional)	Add special tokens for model use (e.g., [CLS], [SEP]).	The internationalization of user-friendly interfaces ...	[CLS], the, inter, ##national, ##ization, of, user, -, friendly, inter, ##faces, [SEP], helps, organizations, ...
5 Final Subword Sequence	Sequence of subword tokens ready for model input.	The internationalization of user-friendly interfaces ...	[CLS], the, inter, ##national, ##ization, of, user, -, friendly, inter, ##faces, [SEP], helps, organizations, communicate, better, .]



BEST PRACTICES

- ✓ Use subword tokenisation for morphologically rich languages or domain-specific terms.
- ✓ Ensure consistent tokenisation across training and inference.
- ✓ Understand your model's tokenizer (e.g., BPE, WordPiece).
- ✓ Keep vocabulary size balanced for performance and coverage.
- ✓ Always inspect how domain terms are split.



ADVANTAGES

- ✓ **Handles rare words:** Breaks them into known pieces.
- ✓ **Smaller vocabulary:** Reduces out-of-vocabulary (OOV) rate.
- ✓ **Better generalisation:** Captures patterns in word parts (prefixes, suffixes, roots).
- ✓ **Efficient:** Shorter sequences than character-level models, more robust than word-level models.

EXAMPLE COMPARISON

Word-level Tokenisation	Subword Tokenisation (BPE)
[the, internationalization, of, user-friendly, interfaces, helps, organizations, communicate, better, .]	[the, inter, ##national, ##ization, of, user, -, friendly, inter, ##faces, helps, organizations, communicate, better, .]
✗ Fails if “internationalization” is unknown.	✓ Can represent even unseen or complex words.



KEY TAKEAWAY

Subword tokenisation gives models the **flexibility** to understand and generate words they haven't seen before by learning from **smaller meaningful pieces**.

WHY TOKENISATION MATTERS FOR MODERN LLMS

Large Language Models (LLMs) do not read text the way humans do. They process text as tokens (words, subwords or pieces). The way we tokenise text has a big impact on how models **learn**, **understand** and **generate** language.

KEY REASONS TOKENISATION MATTERS

1



Handles Any Word

Subword tokenisation breaks unknown or rare words into meaningful pieces, so models can still understand them.

2



Reduces Vocabulary Size

Smaller, well-chosen vocabularies make training faster and more memory efficient.

3



Improves Generalisation

Models learn patterns at the subword level (prefixes, suffixes, roots), helping them generalise better.

4



Better for Multilingual Text

Subword tokenisation works across many languages, even with different scripts and morphology.

5



Efficient Computation

Fewer tokens per sentence (with good tokenisation) reduce sequence length and speed up training and inference.

6



Improves Output Quality

Good tokenisation leads to more accurate, fluent and context-aware text generation.

EXAMPLE: SAME TEXT, DIFFERENT TOKENISATION APPROACHES

Example Text	Word-level Tokenisation (Simple, not used in LLMs)	Subword Tokenisation (BPE) (Used in most LLMs)	Character-level Tokenisation (Not efficient)
	Internationalization NLP makes powerful !	Intern ational ization make s NLP power ful !	I n t e r n a t i o n a l i z a t i o n m a k e s . N L P p o w e r f u l !
	✗ Fails for unknown or rare words	✓ Handles any word, captures meaningful pieces	✗ Too many tokens, slower and less efficient



KEY TAKEAWAYS

- ✓ Tokenisation is the foundation of how LLMs see and understand text.
- ✓ Better tokenisation = better learning, better understanding, better results.
- ✓ Subword tokenisation is essential for real-world, open-domain applications.
- ✓ Always choose or design a tokenizer that fits your data and goals.



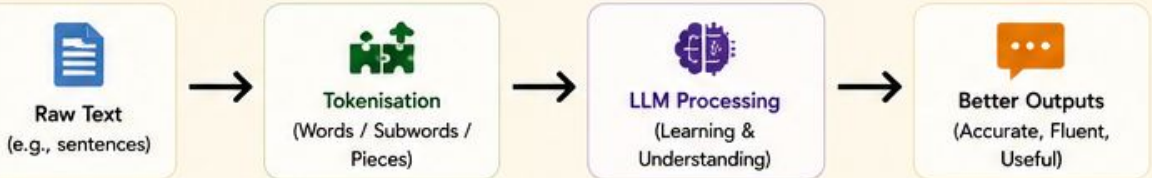
REAL-WORLD IMPACT

- ✓ Enables LLMs to understand niche, technical or domain-specific terms.
- ✓ Supports code, maths, names, slang, typos and multilingual content.
- ✓ Used in models like GPT, LLaMA, Gemini, Mistral, etc.
- ✓ Good tokenisation directly improves user experience and model reliability.



BOTTOM LINE

Tokenisation is not just a technical step — it's the **bridge** between raw text and **intelligent models**.



TOKENISATION CHALLENGES

Tokenisation is essential, but not simple. Different languages, domains and text types introduce several challenges that **affect model performance and reliability**.

CHALLENGE	WHY IT HAPPENS	EXAMPLE	IMPACT
 Out-of-Vocabulary (OOV) Words	Words not seen during training are broken into many pieces or unknown tokens.	"blockchain" → [block, ##chain] or [UNK] in word-level models	• Information loss • Poor understanding • Lower accuracy
 Handling Punctuation and Symbols	Punctuation (., !, ?), symbols (@, #, \$), and emojis can be tokenised inconsistently.	"Great job!!! 😊" → [great, job, !, !, !, @] or [great, job!!!] depending on tokenizer	• Inconsistent outputs • Harder comparison • Noisy representations
 Contractions and Informal Language	Contractions and slang are common in real-world text but tricky to split.	"can't" → [can, 't] or [can't] "gonna" → [go, nna] or [gonna]	• Meaning ambiguity • Affects sentiment & intent analysis
 Multilingual Text	Languages differ in script, word boundaries and morphology.	Chinese: "人工智能很有趣" → [人, 工, 智, 能, 很, 有, 趣] or [人工, 智能, 很, 有趣]	• Inconsistent granularity • Cross-lingual errors • More data needed
 Domain-Specific Terms and Jargon	Technical, medical or legal terms may be rare or very long.	"Electroencephalogram" → [electro, ence, phal, o, gram]	• Fragmented meaning • More tokens • Slower computation
 Inconsistent Tokenisation Across Tools/Models	Different tokenizers use different rules and vocabularies.	Same text → different tokens in BERT vs GPT vs LLaMA	• Hard to reproduce results • Comparison difficulties • Integration issues



BEST PRACTICES

- ✓ Always explore your tokenizer on sample text from your domain.
- ✓ Use subword tokenisation (BPE/WordPiece) for robustness.
- ✓ Add domain-specific vocabulary if needed.
- ✓ Keep preprocessing consistent across training and inference.
- ✓ Evaluate tokenisation quality as part of your pipeline.



THINGS TO REMEMBER

- No single tokenization is perfect.
- Good tokenization balances granularity, coverage and efficiency.
- Understand your data and choose (or build) the tokenizer that fits your task.
- Always test how tokenization affects model performance.



KEY TAKEAWAY

Tokenisation challenges arise from language diversity, informal text, rare words and domain complexity. Understanding and addressing these challenges leads to more **robust**, **accurate** and **fair** NLP systems.

WHAT ARE STOP WORDS?

Stop words are common words that appear frequently in a language but carry **little meaning** by themselves. They are often removed during text preprocessing to **reduce noise, improve efficiency** and focus on the **important content words**.



Example

Original Sentence: "The quick brown fox jumps over the lazy dog."

After removing stop words: "quick brown fox jumps lazy dog"

COMMON ENGLISH STOP WORDS (Examples)

a	an	the	and	or	but	if	because
in	on	at	to	from	by	for	with
is	are	was	were	be	been	being	have
has	had	do	does	did	not	so	such
this	that	these	those	i	you	he	she
it	we	they	my	your	his	her	our

BEFORE vs AFTER REMOVING STOP WORDS

Type	Original Text	After Stop Word Removal	Effect
Sentence	"In this study, we analyze the effect of stress on student performance."	"study, analyze, effect, stress, student, performance"	Cleaner, shorter and more informative.
Document	Contains many common words (the, is, and, of, etc.).	Only content-bearing words (nouns, verbs, adjectives, etc.).	Better representation of the actual content.



KEY TAKEAWAY

Stop words are common but not very informative. Removing them helps NLP systems work more **efficiently** and focus on what really matters — the **meaningful content**.



WHY REMOVE STOP WORDS?



Reduces vocabulary size and dimensionality.



Improves information retrieval and search performance.



Focuses on meaningful content words.



Speeds up processing and reduces storage.



Improves model performance in many NLP tasks.



BEST PRACTICES

- ✓ Use a standard stop word list for your language (e.g., NLTK, spaCy).
- ✓ Customize the list for your domain if needed.
- ✓ Do not remove stop words if they carry meaning in your task (e.g., sentiment, question detection).
- ✓ Always experiment and evaluate the impact on your task.



THINGS TO REMEMBER

- Stop words vary by language.
- Some stop words may be important in specific contexts.
- Removal is optional and task-dependent.



POTENTIAL LIMITATIONS

- May remove useful words in some tasks.
- Can affect grammatical understanding.
- Not always beneficial for short texts.



EXAMPLES OF USE CASES

- Information Retrieval
- Topic Modeling
- Text Classification
- Document Clustering

REMOVING STOP WORDS

Removing stop words is the process of filtering out common words that appear frequently in a language but carry little meaning. This helps **reduce noise, improve efficiency,** and **focus on the informative words** in the text.



Example

Original Sentence: "The students in the class are working on their projects."
After removing stop words: "students class working projects"

HOW STOP WORD REMOVAL WORKS

STEP 1 Tokenisation	STEP 2 Compare	STEP 3 Remove Stop Words	STEP 4 Result
Split the sentence into tokens (words).	Compare each token with the stop word list.	Remove the tokens that are in the stop word list.	The cleaned text contains only meaningful words.
The students in the class are working on their projects	Keep students class working projects Remove (Stop Words) The, in, the, are, on, their	students class working projects	"students class working projects"



COMMON ENGLISH STOP WORDS (Examples)

a, an, the, and, or, but, if, because, in, on, at, to, from, by, for, with, is, are, was, were, be, been, being, have, has, had, do, does, did, not, so, such, this, that, these, those, i, you, he, she, it, we, they, my, your, his, her, our...

BEFORE vs AFTER STOP WORD REMOVAL

Type	Before Removal	After Removal	Effect
Sentence	"The results of the experiment show that the model is accurate."	results experiment show model accurate	Cleaner, shorter and more focused.
Document	Contains many common words (e.g., the, is, and, of, etc.).	Contains only informative content words.	Better representation of the actual content.



BENEFITS

- Reduces vocabulary size and dimensionality.
- Improves efficiency of NLP models.
- Enhances information retrieval and search.
- Focuses on meaningful content words.
- Speeds up processing and reduces storage.



BEST PRACTICES

- Use a standard stop word list (e.g., NLTK, spaCy).
- Customize the list for your domain and task.
- Do not remove stop words in tasks where they carry meaning (e.g., sentiment, questions).
- Always evaluate the impact on performance.



POTENTIAL LIMITATIONS

- May remove useful words in some contexts.
- Can affect grammatical understanding.
- Not always beneficial for short texts.



KEY TAKEAWAY

Removing stop words helps NLP systems focus on the most informative words, leading to more **efficient, accurate** and **effective** text processing.



Raw Text



Tokenisation



Remove Stop Words



Cleaned Text

SHOULD WE ALWAYS REMOVE STOP WORDS?

Not always. While removing stop words can improve efficiency and reduce noise, there are many situations **where stop words carry important meaning** and should be kept.



Example

Query: "Where is the nearest bank?"

If we remove stop words → "nearest bank"

Meaning changes! The word **"where"** is crucial here.



BENEFITS OF REMOVING STOP WORDS

- Reduces vocabulary size and dimensionality.
- Improves efficiency and speeds up processing.
- Enhances information retrieval and search performance.
- Helps focus on the most informative content words.
- Reduces storage and computational cost.

IT DEPENDS ON THE TASK!

REMOVE STOP WORDS WHEN...



The task focuses on **content and topic**
(e.g., document classification, clustering, topic modeling)



Stop words do not affect the meaning
(e.g., "the, is, and" in general text)



Improving efficiency and reducing noise
is a priority.



You are working with **bag-of-words**,
TF-IDF, or similar models.

KEEP STOP WORDS WHEN...



The task depends on the **meaning of the sentence**.
(e.g., sentiment analysis, NLI, QA)



Stop words affect the **grammar or context**.
(e.g., "not", "never", "no")



The task involves **question words or location/time information**.
(e.g., where, when, why, how)



You are using **models that rely on word order and structure** (e.g., transformers).



COMMON STOP WORDS (Examples)

a, an, the, and, or, but, if, because, in, on, at, to, from, by, for, with, is, are, was, were, be, been, being, have, has, had, do, does, did, not, so, such, this, that, these, those, i, you, he, she, it, we, they, my, your, his, her, our...



BEST PRACTICES

- Understand your task and data before deciding.
- Test both options: with and without stop words.
- Use domain-specific stop word lists when needed.
- Evaluate the impact on model performance.

EXAMPLES: KEEP vs REMOVE STOP WORDS

NLP Task	Keep Stop Words?	Why?
 Information Retrieval	Sometimes	Keep question/locations words (where, when, how).
 Sentiment Analysis	Yes	Words like "not", "no" change the sentiment.
 Natural Language Inference	Yes	Stop words help preserve sentence meaning.
 Question Answering	Yes	Question words are essential to understand the query.
 Text Classification	No	Topic is determined by content words, not stop words.
 Clustering / Topic Modeling	No	Focus on meaningful words that represent topics.



KEY TAKEAWAY

Removing stop words is helpful in many cases, but not a universal rule. Always consider the **task, context, and meaning** before deciding.

EXAMPLE: HOW REMOVING STOP WORDS CAN CHANGE MEANING



Stop words may seem unimportant, but in many cases they carry essential meaning. Removing them without understanding context can lead to **incorrect** or **misleading** results.

#	Original Sentence	Stop Words Removed	After Removing Stop Words	What Changes?	Why It Matters
1	Where is the nearest bank?	where, is, the	nearest bank?	⚠ The question word “where” is lost. Now it is just a phrase, not a question .	i “Where” indicates location. Critical for answering correctly.
2	She is not happy.	she, is, not	happy.	⚠ Negation “not” is removed. Meaning becomes opposite .	i Negation changes sentiment and the overall meaning.
3	I agree with you.	i, with, you	agree.	⚠ The object of agreement is lost. Now it’s unclear who is agreed with.	i Removes important relational information.
4	He went to the store yesterday.	he, went, to, the, yesterday	store.	⚠ Time and action are lost. Only the object remains.	i Loses key context (when and what happened).
5	It is possible to solve this problem.	it, is, to, this	possible solve problem.	⚠ Grammatical structure is broken. Meaning becomes unclear.	i Alters grammar and makes the sentence harder to understand.
6	I saw him because he was late.	i, saw, him, because, he, was	late.	⚠ Cause and subject are removed. Only “late” remains.	i Complete loss of context and relationship.



KEY TAKEAWAY

Stop words are not always irrelevant. They can carry crucial grammatical, relational, temporal, negation, and interrogative information. Always consider the **task** and **context** before removing them.



REMEMBER

Removing stop words blindly can lead to loss of meaning and wrong results.

WHAT IS STEMMING?

Stemming is a text normalization technique that reduces words to their **base or root form** by removing prefixes and suffixes using **heuristic (rule-based)** methods. It helps reduce vocabulary size and treats different forms of the same word as one.



Example

The words studying, studies, and studied all refer to the same concept. Stemming reduces them to a common root: **studi**.

HOW STEMMING WORKS		
STEP	DESCRIPTION	EXAMPLE
1  Input Word	Take the original word.	studies
2  Remove Affixes	Apply rules to remove common prefixes/suffixes.	studies → studi
3  Handle Exceptions	Apply additional rules to handle irregular forms.	better → better (no change)
4  Output Stem	Return the base (stem) of the word.	studi



EXAMPLES

Original Word	Stem
running	run
jumps	jump
happily	happi
relational	relat
fairness	fair
studies	studi
connected	connect
better	better



BEST PRACTICES

- ✓ Stemming is language-dependent.
- ✓ Use appropriate stemmer for your language (e.g., Porter, Snowball).
- ✓ Check stems manually to avoid over-stemming.
- ✓ For modern NLP tasks, consider lemmatization instead of stemming when accuracy is critical.



COMMON STEMMERS

- Porter Stemmer (English)
- Snowball Stemmer (multiple languages)
- Lancaster Stemmer



ADVANTAGES



Reduces vocabulary size



Improves information retrieval



Speeds up processing and reduces storage



Groups similar words together



KEY TAKEAWAY

Stemming simplifies words to their root forms, helping NLP systems work more **efficiently** by reducing variation in word forms.

WHAT IS LEMMATISATION?

Lemmatisation is a text normalization technique that reduces words to their **dictionary form**, called the **lemma**, considering the word's **part of speech (POS)** and **meaning**. It is more accurate than stemming because it uses vocabulary and morphological analysis.



Example

The words studying, studies, studied all refer to the same concept. Lemmatisation reduces them to the dictionary form: **study**.

HOW LEMMATISATION WORKS		
STEP	DESCRIPTION	EXAMPLE (Word: studied)
1  Input Word	Take the original word.	studied
2  Analyse	Determine the part of speech (POS) and morphological features.	POS: Verb (Past Simple) Features: Tense=Past, VerbForm=Participle
3  Lookup & Apply Rules	Use a lexicon and morphological rules to find the dictionary form (lemma).	Mapping: studied → study
4  Output Lemma	Return the lemma (base form).	study



BENEFITS

- Reduces vocabulary size effectively.
- Improves information retrieval and search.
- Enhances text understanding and NLP performance.
- Handles irregular forms and preserves meaning.



BEST PRACTICES

- Use a reliable lemmatiser (e.g., spaCy, NLTK, Stanza).
- Always provide or detect the correct POS tags.
- Lemmatisation works best with clean, well-tokenised text.
- For high precision tasks, prefer lemmatisation over stemming.



WHEN TO USE LEMMATISATION

- When meaning and accuracy are important.
- In search engines, chatbots, information extraction.
- For tasks involving grammar, semantics or knowledge bases.



KEY TAKEAWAY

Lemmatisation reduces words to **meaningful** dictionary forms using language knowledge and context, resulting in more **accurate** and **semantically correct** NLP applications.



Remember: Lemmatise with **understanding**, not just by removing endings!

STEMMING vs LEMMATISATION		
STEMMING		LEMMATISATION
	Uses simple rule-based heuristics (prefix/suffix removal).	 Uses vocabulary and morphological analysis with POS and context.
	May produce non-words that are not in dictionary.	 Produces valid dictionary words (lemmas).
	Fast and language-independent (mostly).	 Slightly slower but more accurate and meaningful.
	Less accurate; may lose meaning.	 More accurate; preserves meaning and context.

EXAMPLES		
Original Word	Part of Speech	Lemma (Dictionary Form)
running	Verb (Present Participle)	run
better	Adjective (Comparative)	good
mice	Noun (Plural)	mouse
went	Verb (Past)	go
children	Noun (Plural)	child
was	Verb (Past)	be

STEMMING & LEMMATISATION

Stemming and lemmatisation reduce words to their base or root form. This groups different forms of a word together, improving consistency and retrieval.

Goal: to normalise words so that similar words are treated as the same term.

STEMMING (Rule-based)			LEMMATISATION (Dictionary/Model-based)		
Word (Raw)	Stemmed	Explanation	Word (Raw)	Lemmatized	Explanation
 running	run	Suffix removed (-ing)	 running	run	Correct base form (verb)
 jumps	jump	Suffix removed (-s)	 jumps	jump	Correct base form (verb)
 studies	studi	Suffix removed (-es)	 studies	study	Correct base form (noun/verb)
 better	better	No change (may be incorrect in some cases)	 better	good	Mapped to dictionary base form
 relational	relat	Suffix removed (-ional)	 relational	relation	Correct base form (noun)

 **NOTE** Use lemmatisation when you need correct base words (e.g., for search, QA, summarisation). Stemming may be useful for quick tasks where exact meaning is less critical.

 **KEY TAKEAWAY**

Stemming and lemmatisation reduce words to their base forms, helping your model focus on the core meaning and improve accuracy.



WHY IT MATTERS

- ✓ Groups word variations together
- ✓ Improves search and matching
- ✓ Reduces vocabulary size
- ✓ Helps models generalise better
- ✓ Improves overall NLP performance

STEMMING vs LEMMATISATION	
STEMMING	LEMMATISATION
• Uses simple rules • May produce non-words (e.g., “studi”) • Faster • Language independent	• Uses dictionary/grammar • Produces meaningful words (e.g., “study”) • Slower • Language dependent

 **MINI DEMO (Python)**

```
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

ps = PorterStemmer()
lem = WordNetLemmatizer()

words = ["running", "jumps", "studies", "better"]

print("Stemmed: ", [ps.stem(w) for w in words])
print("Lemmatized: ", [lem.lemmatize(w, pos='v') for w in words])

# Output: Stemmed: ['run', 'jump', 'studi', 'better']
#         Lemmatized: ['run', 'jump', 'study', 'be']
```

NORMALISATION: HANDLING VARIATIONS

Normalisation deals with **different ways** of writing the same thing.
It standardises text for **consistency** in NLP tasks.

Goal: to make text uniform by applying rules to variations in spelling, case, numbers and more.

COMMON TYPES OF NORMALISATION				
Type	Purpose	Example (Before)	Example (After)	Explanation
 Case Normalisation	Make all text the same case (usually lowercase)	NLP Is Amazing!	nlp is amazing!	Lowercase makes matching easier
 Number Normalisation	Convert numbers to a standard format	I have 2 dogs.	i have 2 dogs.	Keeps numbers consistent
 Contraction Expansion	Expand contractions to full words	I can't wait.	i cannot wait.	Improves clarity for models
 Spelling Normalisation	Correct common spelling mistakes	recieve the file	receive the file	Reduces noise from misspellings
 Whitespace Normalisation	Remove extra spaces, new lines, tabs	This is good. (New lines, tabs)	this is good.	Cleaner text for better processing
 URL / Email Normalisation	Standardise URLs and email addresses	Visit HTTP://UOM.AC.MU	visit http://uom.ac.mu	Uniform links for consistency



NOTE

Choose normalisation steps based on your task and data.
Over-normalisation can remove useful information.



KEY TAKEAWAY

Normalisation ensures that different forms of the same text are treated uniformly, leading to **cleaner data** and **better NLP performance**.



WHY IT MATTERS

- ✓ Reduces data sparsity
- ✓ Improves matching and retrieval
- ✓ Enhances model accuracy
- ✓ Essential for real-world NLP applications
- ✓ Makes downstream tasks more reliable

OTHER VARIATIONS TO HANDLE



Diacritics: café → cafe



Hyphenation: state-of-the-art → state of the art



Abbreviations: e.g. → for example



Date Normalisation: 05/08/2026 → 2026-08-05



Currency Normalisation: Rs. 1,000 → 1000 MUR



Repeated Characters: coool → cool



MINI DEMO (Python)

```
import re
text = " I CAN'T wait!!! Visit HTTP://UOM.AC.MU on 05/08/2026. "
# Lowercase
text = text.lower()
# Expand contraction
text = text.replace("can't", "cannot")
# Remove extra spaces
text = re.sub(r'\s+', ' ', text).strip()
# Output
print(text)
# Output: i cannot wait!!! visit http://uom.ac.mu on 05/08/2026.
```

TEXT PREPROCESSING PIPELINE: PUTTING IT ALL TOGETHER

Text preprocessing involves multiple steps to convert raw text into clean, meaningful and consistent data that machines can understand.

Each step builds on the previous one to improve the quality of the text.

Goal: to prepare text data for effective NLP model training and evaluation.

THE TEXT PREPROCESSING PIPELINE		
	1. Raw Text	Original text as collected from a source. "Wow!!! NLP is awesome, let's learn more about tokenisation."
↓		
	2. Tokenisation	Split text into tokens (words or subwords). ['Wow', 'NLP', 'is', 'awesome', 'let's', 'learn', 'more', 'about', 'tokenisation', '.']
↓		
	3. Remove Punctuation	Remove punctuation and special characters. ['Wow', 'NLP', 'is', 'awesome', 'lets', 'learn', 'more', 'about', 'tokenisation']
↓		
	4. Remove Stop Words	Remove common stop words (e.g., is, the, and, etc.). ['Wow', 'NLP', 'awesome', 'lets', 'learn', 'more', 'tokenisation']
↓		
	5. Stemming / Lemmatisation	Reduce words to their base or root form. ['wow', 'nlp', 'awesom', 'let', 'learn', 'more', 'tokenis']
↓		
	6. Normalisation	Standardise case, numbers, spelling and other variations. ['wow', 'nlp', 'awesom', 'let', 'learn', 'more', 'tokenis']
↓		
	7. Clean Text (Output)	Final clean text ready for NLP tasks. wow nlp awesom let learn more tokenis



NOTE: The exact steps and order may vary depending on the task, dataset and model. Always inspect intermediate results to ensure information is not lost unintentionally.



KEY TAKEAWAY

A well-designed preprocessing pipeline transforms messy, raw text into high-quality data that helps NLP models learn **better** and deliver more **accurate results**.



WHY IT MATTERS

- ✓ Improves data quality and consistency
- ✓ Reduces noise and irrelevant information
- ✓ Enhances model accuracy and performance
- ✓ Speeds up training and inference
- ✓ Makes downstream NLP tasks more reliable



BEST PRACTICES

- Understand your data and task before preprocessing.
- Apply only the steps that are necessary.
- Keep a copy of raw data for reference.
- Use domain-specific stop words when needed.
- Evaluate the impact of each step on performance.
- Document your preprocessing pipeline clearly.



MINI DEMO (Python)

```
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

text = "Wow!!! NLP is awesome, let's learn more about tokenisation."
# Lowercase
text = text.lower()
# Remove punctuation
text = re.sub(r'[a-z\s]', ' ', text)
# Tokenise
tokens = text.split()
# Remove stop words
stop_words = set(stopwords.words('english'))
tokens = [w for w in tokens if w not in stop_words]
# Lemmatise
lemmatizer = WordNetLemmatizer()
tokens = [lemmatizer.lemmatize(w) for w in tokens]
result = ' '.join(tokens)
# Output: wow nlp awesome let learn more tokenisation
```

FROM RAW TEXT TO CLEAN DATA: AN EXAMPLE WALKTHROUGH

Let's see how a raw text goes through the full preprocessing pipeline step by step. Each step improves the text quality and prepares it for NLP models.

Goal: Transform messy, raw text into clean, consistent data.

STEP-BY-STEP TRANSFORMATION		
Step	Operation	Output (Result)
 1. Raw Text	Original text as collected.	Wow!!! NLP is awesome, let's learn more about tokenisation.
 2. Tokenisation	Split into tokens (words).	['Wow!!!', 'NLP', 'is', 'awesome', 'let's', 'learn', 'more', 'about', 'tokenisation']
 3. Remove Punctuation	Remove punctuation & special characters.	['Wow', 'NLP', 'is', 'awesome', 'lets', 'learn', 'more', 'about', 'tokenisation']
 4. Remove Stop Words	Remove common stop words (e.g., is, let's, more, about).	['Wow', 'NLP', 'awesome', 'learn', 'tokenisation']
 5. Stemming / Lemmatisation	Reduce words to base or root form.	['wow', 'nlp', 'awesom', 'learn', 'tokenis']
 6. Normalisation	Standardise case, numbers, spelling, etc.	['wow', 'nlp', 'awesom', 'learn', 'tokenis']
 7. Clean Text (Final)	Join tokens into final clean text.	wow nlp awesom learn tokenis



NOTE: The exact result may vary slightly depending on the tools, rules and resources used. Always check intermediate results to avoid removing important information.



KEY TAKEAWAY

A well-executed preprocessing pipeline converts raw text into high-quality, meaningful data that helps NLP models perform **better** and **more reliably**.



WHY IT MATTERS

- ✓ Shows the impact of each preprocessing step
- ✓ Helps in understanding the transformations
- ✓ Builds intuition for real-world NLP tasks
- ✓ Leads to better model accuracy and reliability



WHAT CHANGED?

- **Noise removed:** Extra symbols, punctuation, stop words.
- **Standardised:** Case, spelling, and word forms.
- **Meaning preserved:** Core information remains.
- **Model-ready:** Text is now consistent and compact.



MINI DEMO (Python)

```
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

text = "Wow!!! NLP is awesome, let's learn more about tokenisation."
# 1. Lowercase
text = text.lower()
# 2. Remove punctuation
text = re.sub(r'[^a-z\s]', ' ', text)
# 3. Tokenise
tokens = text.split()
# 4. Remove stop words
stop_words = set(stopwords.words('english'))
tokens = [w for w in tokens if w not in stop_words]
# 5. Lemmatise
lem = WordNetLemmatizer()
tokens = [lem.lemmatize(w) for w in tokens]
# 6. Join
clean_text = ' '.join(tokens)
# Output: wow nlp awesom learn tokenisation
```

EVALUATING TEXT PREPROCESSING

How do we know if our preprocessing is working well?
We use simple evaluation methods to measure the impact of each step on our text data.
Goal: Quantify improvements in quality, consistency and usefulness of text.

COMMON EVALUATION METRICS				
Metric	What It Measures	How It Is Calculated	Good Result	Why It Matters
 Token Count Reduction	Reduction in number of tokens after preprocessing.	% Reduction = $(1 - \text{New} / \text{Original}) \times 100$	Moderate to High (20% – 60%)	Shows removal of noise and redundancy.
 Unique Token Ratio (UTR)	Proportion of unique tokens in the text.	UTR = Unique Tokens / Total Tokens	Higher is Better (> 0.4)	Indicates richer vocabulary and less repetition.
 Information Retention	Measure of important content retained after preprocessing.	Manually or automatically (compare key entities, keywords or labels)	High Retention (> 90%)	Ensures important information is not lost.
 Stop Word Removal %	Proportion of tokens removed as stop words.	% Stop Words = $\text{Stop Words Removed} / \text{Original Tokens} \times 100$	Depends on Domain (10% – 40%)	Shows how much noise was removed.
 Processing Time	Time taken to preprocess the text.	Measured in seconds for a dataset.	Lower is Better	Important for efficiency and scalability.

 **NOTE:** Ideal values depend on the task, dataset and domain.
Always compare before and after results to understand the impact.

KEY TAKEAWAY

Evaluation helps you measure the **impact** of preprocessing, ensuring your text data is clean, meaningful and **model-ready**.



WHY EVALUATION MATTERS

- ✓ Helps choose the best preprocessing steps.
- ✓ Prevents over-processing (loss of information).
- ✓ Improves model training and prediction.
- ✓ Allows comparison of different approaches.
- ✓ Builds confidence in your data quality.

EXAMPLE: BEFORE vs AFTER PREPROCESSING

Metric	Before	After	Improvement
Total Tokens	15,200	9,100	-40.1%
Unique Tokens	2,150	1,980	-7.9%
Unique Token Ratio	0.141	0.218	+54.6%
Stop Words %	33.2%	0%	-100%
Information Retention*	—	94.3%	High

* Example measured using keyword overlap with original text.



MINI DEMO (Python)

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from collections import Counter

text = "Wow!!! NLP is awesome, let's learn more about tokenisation."
tokens = word_tokenize(text.lower())
stop_words = set(stopwords.words('english'))
tokens_nostop = [w for w in tokens if w not in stop_words]

print("Original Tokens:", len(tokens))
print("After Removing Stopwords:", len(tokens_nostop))
print("Unique Tokens (Before):", len(set(tokens)))
print("Unique Tokens (After):", len(set(tokens_nostop)))
```

Count tokens
Remove stop words
Stats
Stats

TIPS & BEST PRACTICES FOR TEXT PREPROCESSING

Good preprocessing makes your data cleaner, more consistent and more useful.

These practical tips help you make better decisions for your NLP tasks.

Goal: Apply effective and consistent preprocessing that supports accurate NLP models.

✓ DO THIS	
 Understand your task	Choose preprocessing steps based on the goal (e.g., classification, summarisation, translation).
 Know your data	Explore your dataset (size, language, noise, domain) before preprocessing.
 Start simple	Begin with basic steps and add complex ones only if needed.
 Inspect intermediate results	Check the output after each step to avoid losing important information.
 Be consistent	Apply the same preprocessing to train, validation and test data.
 Evaluate impact	Measure the effect of preprocessing on model performance and data quality.
 Document your pipeline	Keep a clear record of steps, tools and parameters for reproducibility.

✗ AVOID THIS	
 Over-processing the text	Removing too much (e.g., punctuation, stop words) can remove important context.
 Ignoring domain specifics	Generic rules may not work well for specific domains or languages.
 Inconsistent processing	Different preprocessing between training and testing leads to poor results.
 Not validating changes	Skipping evaluation can hide negative impact on model performance.
 Hard-coding everything	Make your pipeline configurable and easy to adapt.
 Ignoring edge cases	Handle missing values, empty strings, and unusual characters.
 Focusing only on accuracy	Efficiency, scalability and interpretability are also important.



NOTE: There is no “one size fits all” preprocessing.
Good preprocessing = the right steps for the right data and the right task.



KEY TAKEAWAY

Smart preprocessing leads to **cleaner data**, **better models** and **more reliable results**.



QUICK CHECKLIST

- ✓ Do I understand my task and data?
- ✓ Have I chosen the right preprocessing steps?
- ✓ Have I inspected intermediate outputs?
- ✓ Is the same pipeline applied to all datasets?
- ✓ Have I evaluated the impact on my model?
- ✓ Is my pipeline documented and reproducible?



KEY PRINCIPLES

- Clean, but not too clean.
- Consistent, but not rigid.
- Simple, but effective.
- Evaluate, always.



MINI DEMO (Python)

```
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

text = "Wow!!! NLP is awesome, let's learn more about tokenisation."
# Simple preprocessing pipeline
text = text.lower() # Lowercase
text = re.sub(r'[?a-z\s]', ' ', text) # Remove punctuation
tokens = text.split()
stop_words = set(stopwords.words('english'))
tokens = [w for w in tokens if w not in stop_words] # Remove stop words
lemmatizer = WordNetLemmatizer()
tokens = [lemmatizer.lemmatize(w) for w in tokens] # Lemmatize
clean_text = ' '.join(tokens)
print(clean_text)

# Output: wow nlp awesome learn more tokenisation
```

REAL-WORLD APPLICATIONS OF NLP

Natural Language Processing powers many tools and services we use every day. By converting human language into meaningful data, NLP helps machines understand, generate and interact with text and speech.

Goal: See how NLP is used to solve real-world problems and improve lives.

WHERE NLP IS MAKING AN IMPACT			
 1. Chatbots & Virtual Assistants Answer questions, provide support and automate conversations. Examples: ChatGPT, Siri, Google Assistant	 2. Search Engines & Information Retrieval Understand queries, rank relevant results and return the best answers. Examples: Google Search, Bing	 3. Machine Translation Translate text from one language to another automatically. Examples: Google Translate, DeepL, Microsoft Translator	 4. Text Summarisation Reduce long texts into short, meaningful summaries. Examples: News summaries, research abstracts
 5. Sentiment Analysis Detect opinions, emotions and attitudes in text. Examples: Product reviews, social media monitoring	 6. Spam & Abuse Detection Identify unwanted, harmful or inappropriate content. Examples: Email spam filters, hate speech detection	 7. Document Understanding Extract information from forms, contracts and documents. Examples: Invoices, resumes, IDP (Intelligent Document Processing)	 8. Speech Recognition Convert speech into text for hands-free interaction and accessibility. Examples: Voice typing, transcription tools



NOTE: NLP is transforming the way humans and machines communicate. Understanding text is the first step toward building intelligent systems.



KEY TAKEAWAY

NLP applications are everywhere — from chatbots to search, translation, and beyond. Clean, well-processed text is the **foundation** of all these intelligent systems.



WHY THESE APPLICATIONS MATTER

- ✓ Improve efficiency and productivity
- ✓ Enhance user experience and accessibility
- ✓ Enable data-driven decisions
- ✓ Automate repetitive and time-consuming tasks
- ✓ Create smarter, more human-like systems



CHALLENGES TO CONSIDER

- ✓ **Ambiguity:** Human language is complex and context-dependent.
- ✓ **Bias:** Models can learn and amplify biases in data.
- ✓ **Multilinguality:** Handling many languages and dialects.
- ✓ **Privacy & Ethics:** Protecting sensitive information and user privacy.
- ✓ **Data Quality:** Poor data leads to poor performance.



MINI DEMO (Python) – Simple Sentiment Analysis

```
from textblob import TextBlob
text = "I love this NLP class! It is very interesting and useful."
blob = TextBlob(text)
polarity = blob.sentiment.polarity
if polarity > 0:
    sentiment = "Positive"
elif polarity < 0:
    sentiment = "Negative"
else:
    sentiment = "Neutral"
print("Sentiment:", sentiment, "| Polarity:", polarity)
# Output: Sentiment: Positive | Polarity: 0.62
```

Interpreting Polarity

- 1.0 to 0.2 → **Positive** 😊
 - -0.2 to 0.2 → **Neutral** 😐
 - -1.0 to -0.2 → **Negative** 😞
- (Values range from -1 to 1)